

EECS 4101/5101

Prof. Andy Mirzaian



**Computer Science
and Engineering**

120 Campus Walk

Convex Hull

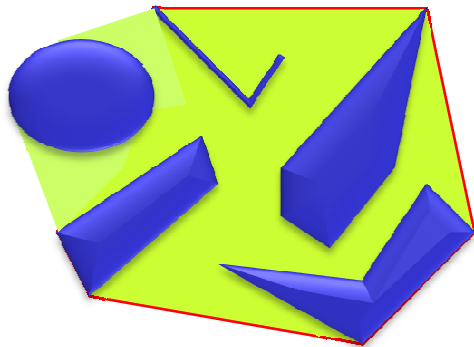
CONVEX HULL

Given $S \subseteq \mathbb{R}^d$ ($d = 1, 2, 3, \dots$)

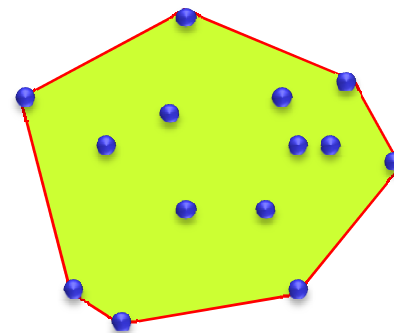
Convex-Hull of S :

$\text{CH}(S)$ = the set of all convex combinations of points in S
= the smallest convex set that contains S
= the intersection of all convex sets that contain S

The rubber-band analogy: Place a rubber-band around S and let it tighten.



Example 1: $|S|$ infinite



Example 2: $|S|$ finite

The Convex Hull Problem

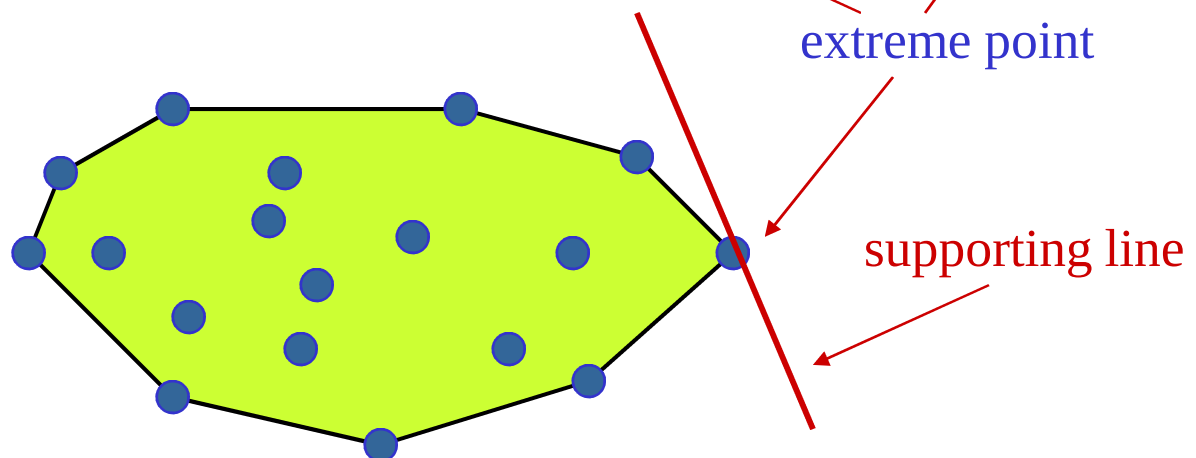
Given a set $S = \{p_1, p_2, \dots, p_n\}$ of n points in $\mathbb{R}^d$, compute $\text{CH}(S)$.

In general, $\text{CH}(S)$ will be a convex polytope whose vertices are a subset of S , called **extreme points** of S .

Dimension $d=1$: trivial, $O(n)$ time.



Dimension $d=2$:

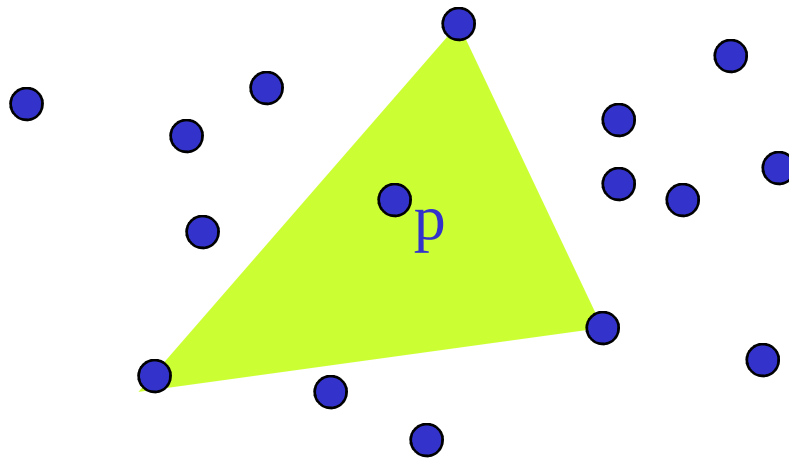


2D Convex Hull: Early development

Caratheodory's Theorem: p is NOT an extreme point
 $\Leftrightarrow p$ is a convex-combination of up to 3 ($=d+1$) other points.

An $O(n^4)$ time algorithm:

- Eliminate non-extreme points by checking each 4-point subset.
- Then order the extreme points around the convex-hull. (How?)



$O(n^4)$ is too slow.

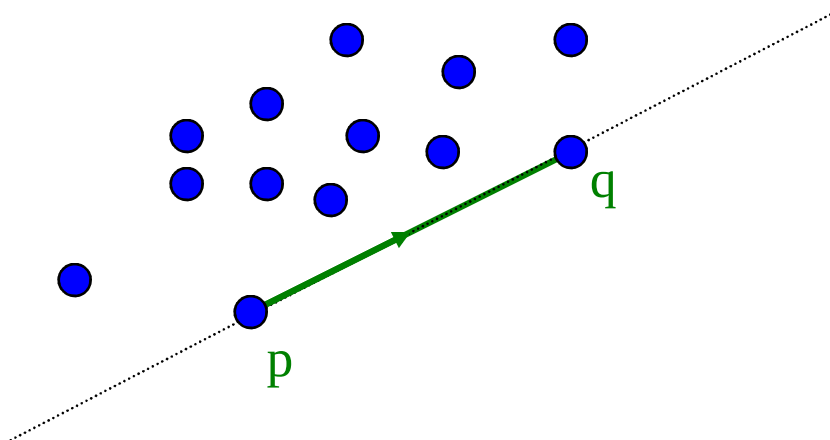
2D CH: An $O(n^3)$ time algorithm

Algorithm CH(P)

- $E \leftarrow \emptyset$ (* edge-list of CH(P) *)
- **for** all ordered pairs $(p,q) \in P \times P, p \neq q$ **do**
- supporting $\leftarrow$ true
- **for** all points $r \in P - \{p,q\}$ **do**
- **if** r is on the right side of $\overrightarrow{pq}$ **then** supporting $\leftarrow$ false
- **if** supporting **then** add directed edge $\overrightarrow{pq}$ to E
- From the (un-ordered) edge-list E , construct the list of vertices of CH(P) in CCW order in $O(n)$ time. (How?)

end

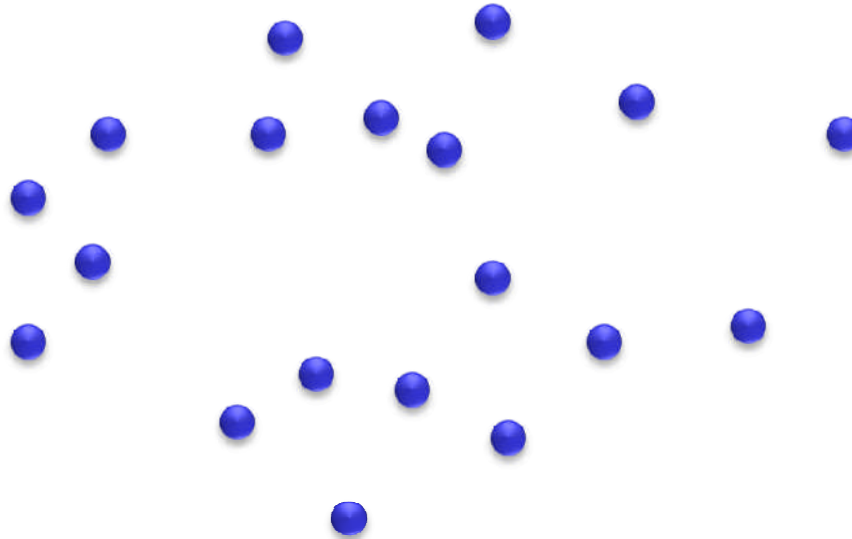
$O(n^3)$ still too slow!



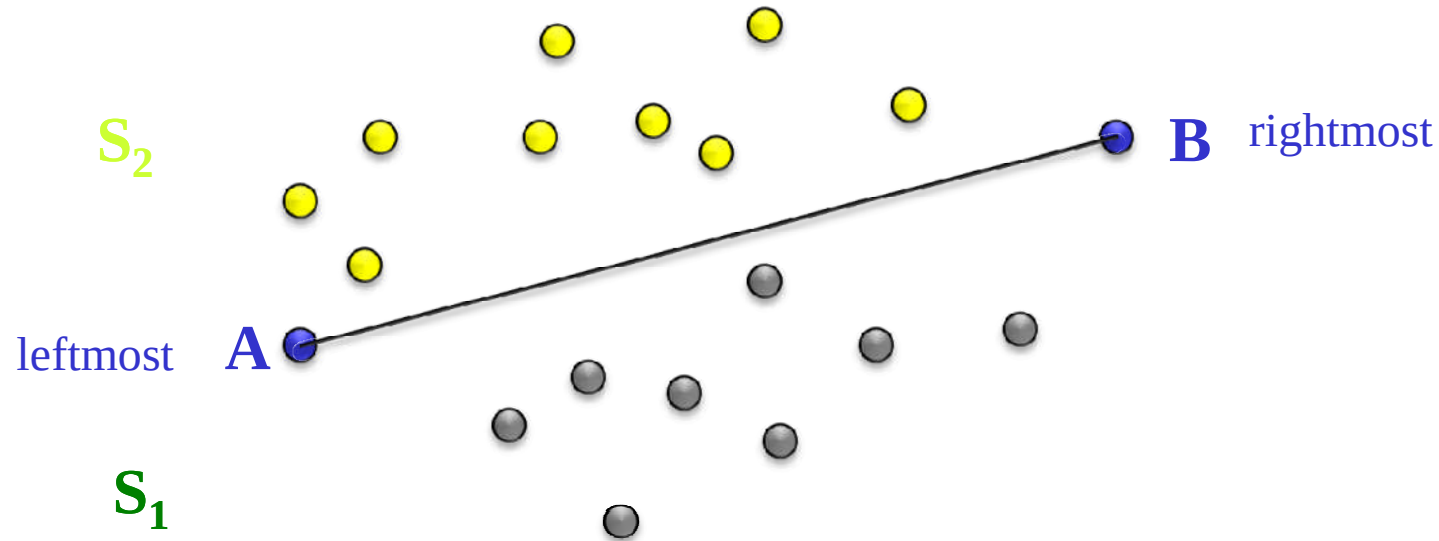
QUICK HULL

(similar to QuickSort - see AAW)

S



QUICK HULL



A & B are extreme points (admit vertical supporting lines)

$S_1 \leftarrow$ points to the right of $\overrightarrow{AB}$

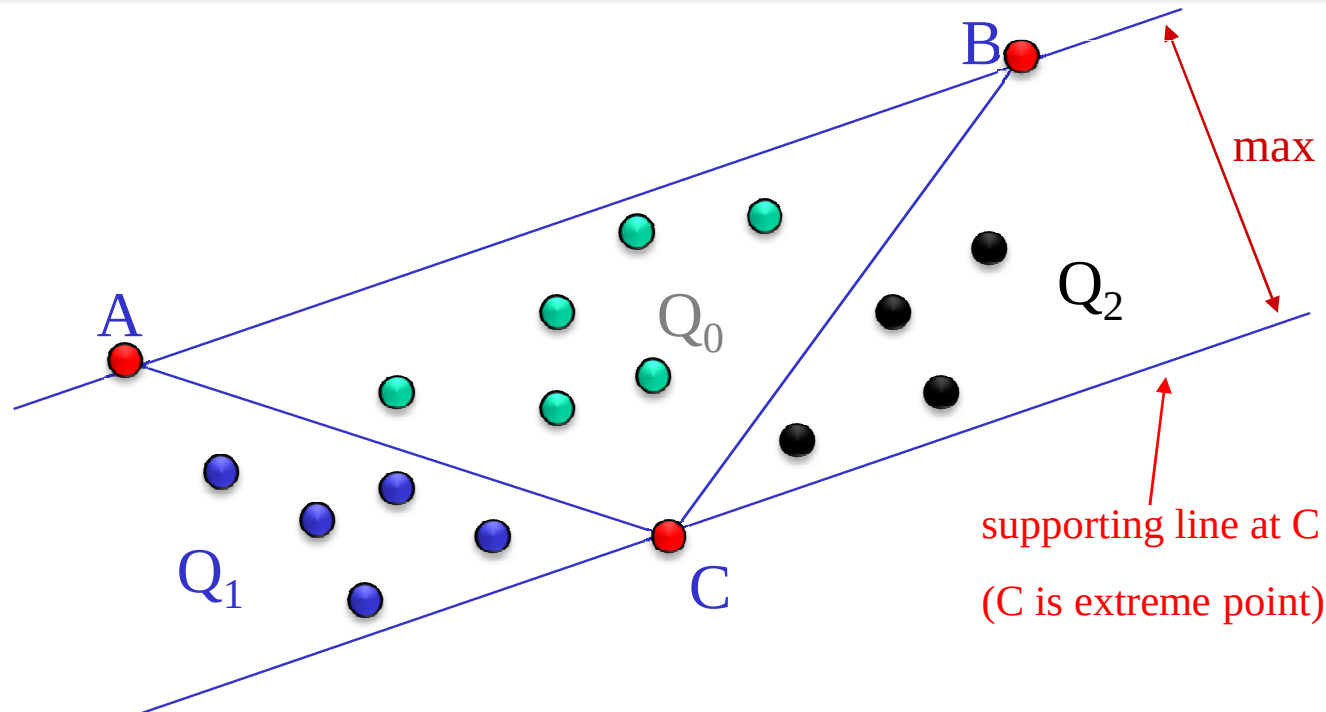
$S_2 \leftarrow$ points to the right of $\overrightarrow{BA}$

Initialize CH to $\overrightarrow{AB}$ followed by $\overrightarrow{BA}$. (uninflated balloon)

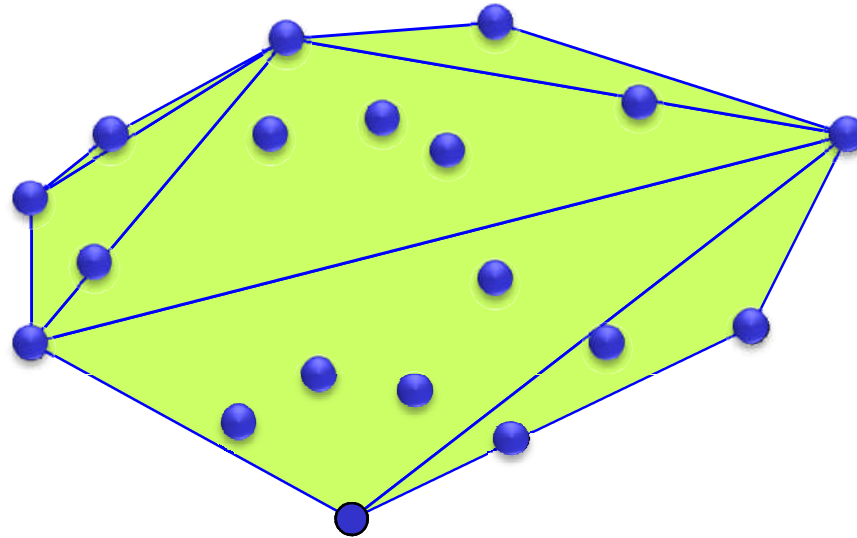
Call FindHull(S_1, A, B) and FindHull(S_2, B, A)

FindHull(P,A,B):

- If P is empty, then return.
- Find (extreme) point $C \in P$ orthogonally farthest from $\overline{AB}$.
- CH update: replace $\overline{AB}$ with $\overrightarrow{AC}$ followed by $\overrightarrow{CB}$. (inflate baloon)
- Partition $P - \{C\}$ into Q_0, Q_1, Q_2 as shown.
- Discard Q_0 inside triangle ABC (by Caratheodory).
- Recursively call FindHull(Q_1, A, C) and FindHull(Q_2, C, B).



QuickHull animation



**$T(n)$ = Time Complexity of FindHull(P,A,B),
where $n=|P|$**

- Find C, Q_0 , Q_1 , Q_2 : $O(n)$ time
- FindHull(Q_1 ,A,C): $T(q)$ time. ($|Q_1| = q$)
- FindHull(Q_2 ,C,B): $T(n-1-q)$ time. ($|Q_2| \leq n-1-q$)

Worst Case:

$$\begin{aligned} T(n) &= \max \{ T(q) + T(n-1-q) + O(n) \mid 0 \leq q \leq n-1 \} \\ &= T(0) + T(n-1) + O(n) \\ &= O(n^2) \end{aligned}$$

Average Case:

On many realistic point distributions
 $\text{Avg}[T(n)] = O(n)$ or close to it.

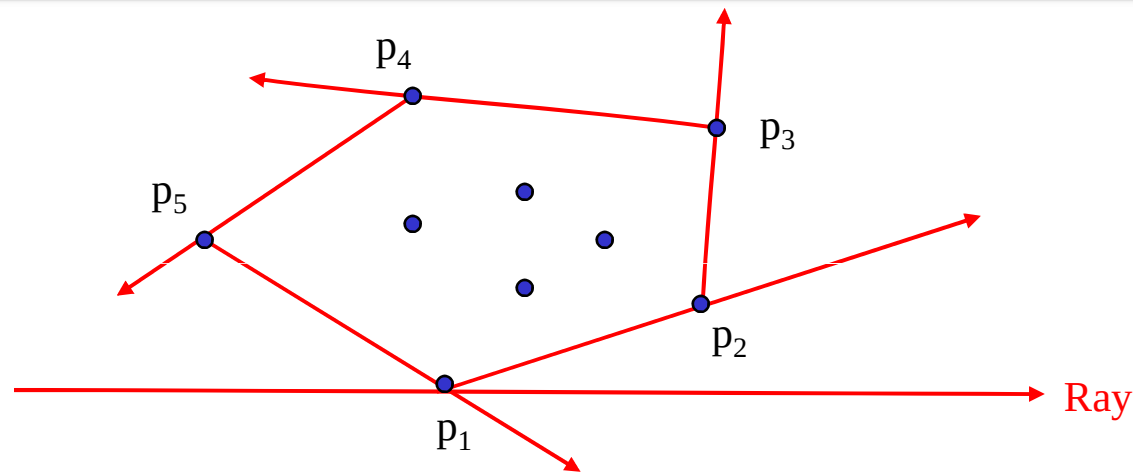
Algorithm Jarvis' March [1973]

{ gift-wrapping method, generalizes to higher dimensions }

Step 1: Let p_1 be the point with minimum y-coordinate (lex.)

Step 2: Anchor ray at current point and rotate to next anchor point.

Repeat.



Output-sensitive: $O(nh)$ time.

n = # input points, h = # hull vertices (output size)
($3 \leq h \leq n$ if $n \geq 3$ and not all points collinear)

Worst-case: $O(n^2)$ time.

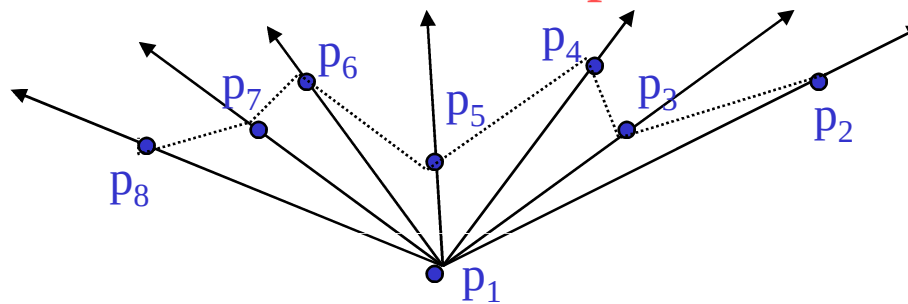
Algorithm Graham-Scan ($p_1 \dots p_n$) [1972]

Step 1: **Polar-Sort** $p_1 \dots p_n$:

Find a lexicographically min point among p_i
(e.g., leftmost-lowest point) and swap with p_1 .

Sort $p_2 \dots p_n$ by their polar angle of rays $\overrightarrow{p_1 p_i}$, $i=2..n$.
(Use Δ -test to simulate comparisons in the sorting.)

$O(n \log n)$



Step 2: Stack $S \leftarrow (p_2, p_1)$ (* p_1 at the bottom *)

for $i \leftarrow 3 \dots n$ **do**

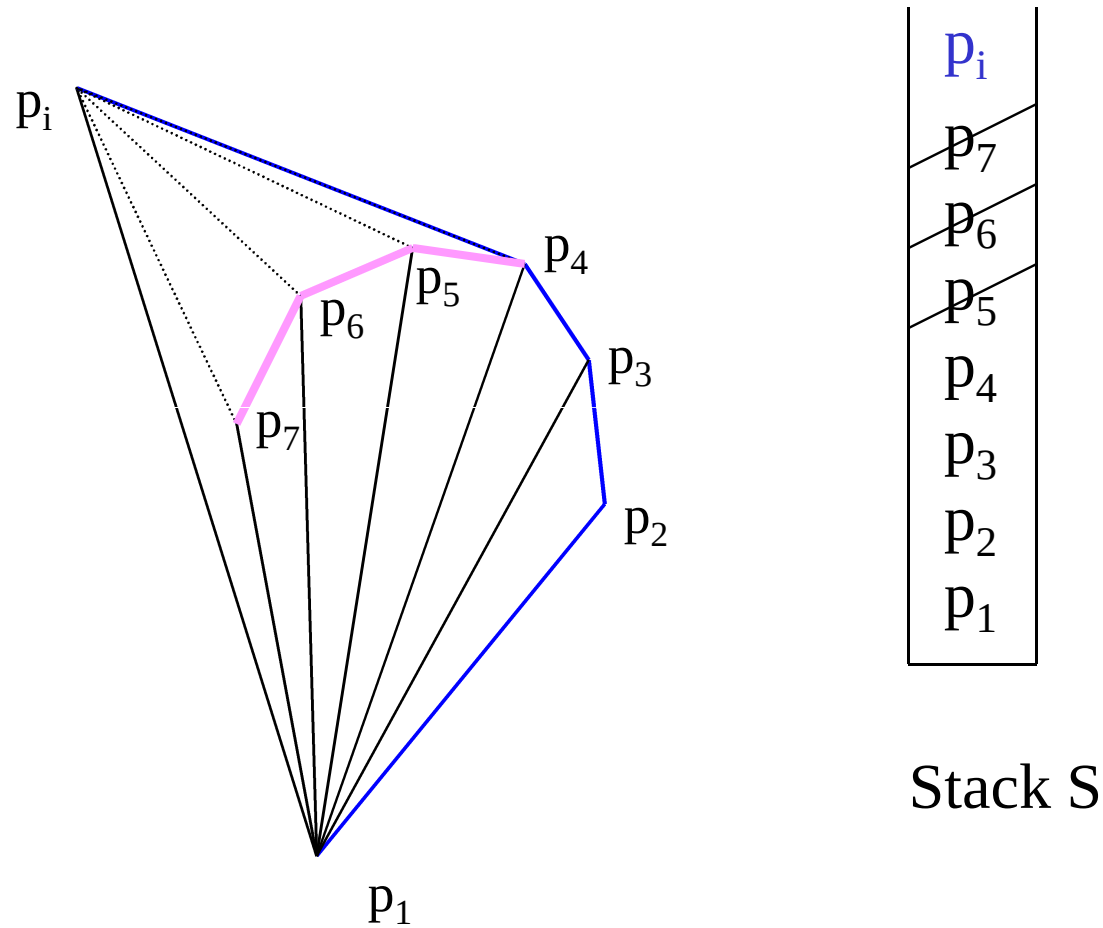
$O(n)$ **while** not $CW(p_i, \text{Top}(S), \text{2ndTop}(S))$ **do** POP(S)
 PUSH(p_i, S)

end-for

Step 3: **return** S (* vertices of $CH(p_1 \dots p_n)$ in CCW order *)

$O(n)$

Graham-Scan : a snapshot of step 2



$\Omega(n \log n)$ **Lower Bound** for 2D Convex Hull Problem

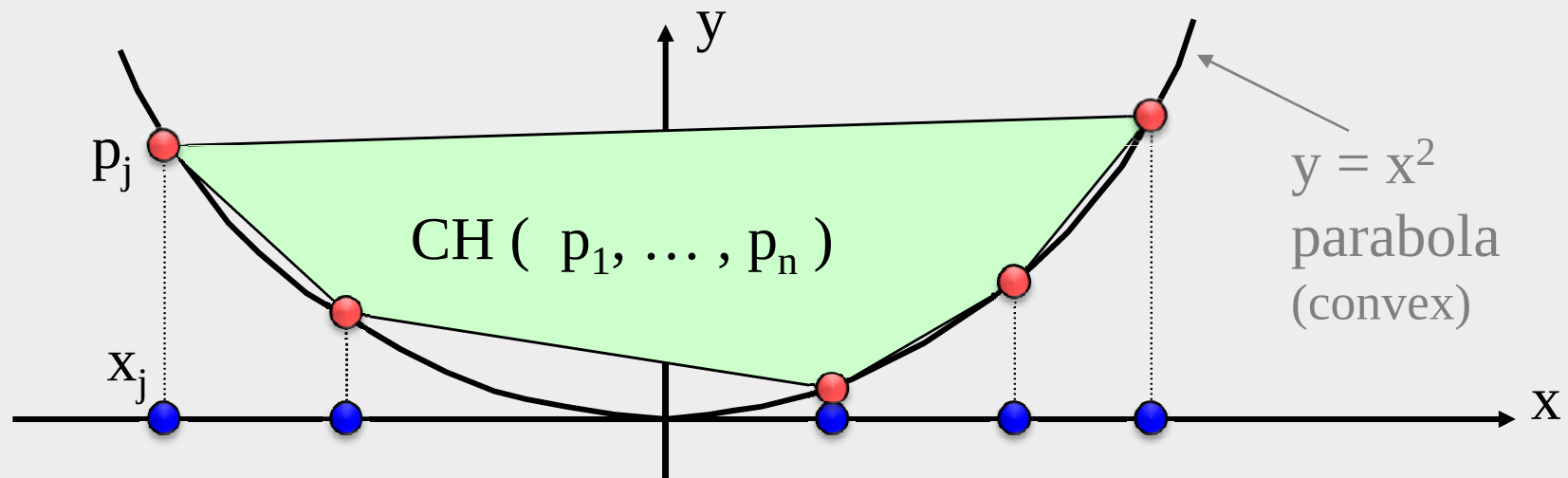
Reduction from Sorting (using the Lifting Method)

Sort: $X = (x_1, x_2, \dots, x_n)$ (n given real numbers on the x-axis)



Step 1: $P \leftarrow \{p_1, p_2, \dots, p_n\}$, where $p_j = (x_j, x_j^2)$, $j=1..n$

Step 2: Compute $CH(P)$



Step 3: $Sorted(X)$ can be obtained from $CH(P)$ in $O(n)$ added time.

$$\Omega(n \log n) = O(n) + T(n) + O(n) \Rightarrow T(n) = \Omega(n \log n).$$

Algorithm Divide-&-Conquer 2D Convex-Hull:

- x-sort the given points $p_1, p_2, \dots, p_n$ (lexicographically).
- Call $CH(\{p_1, p_2, \dots, p_n\})$.

Procedure $CH(S)$:

- **Base**: if $|S| \leq 3$ then return trivial answer.
- **Divide**: Partition S into two (almost) equal halves L and R around the **x-median** of S .
- **Conquer**: Recursively call $CH(L)$ and $CH(R)$.
- **Merge**: Compute common upper/lower bridges of $CH(L)$ and $CH(R)$, and obtain $CH(S)$.

(See next page.)

$T(n) =$

$O(1)$

or

$O(n)$

+

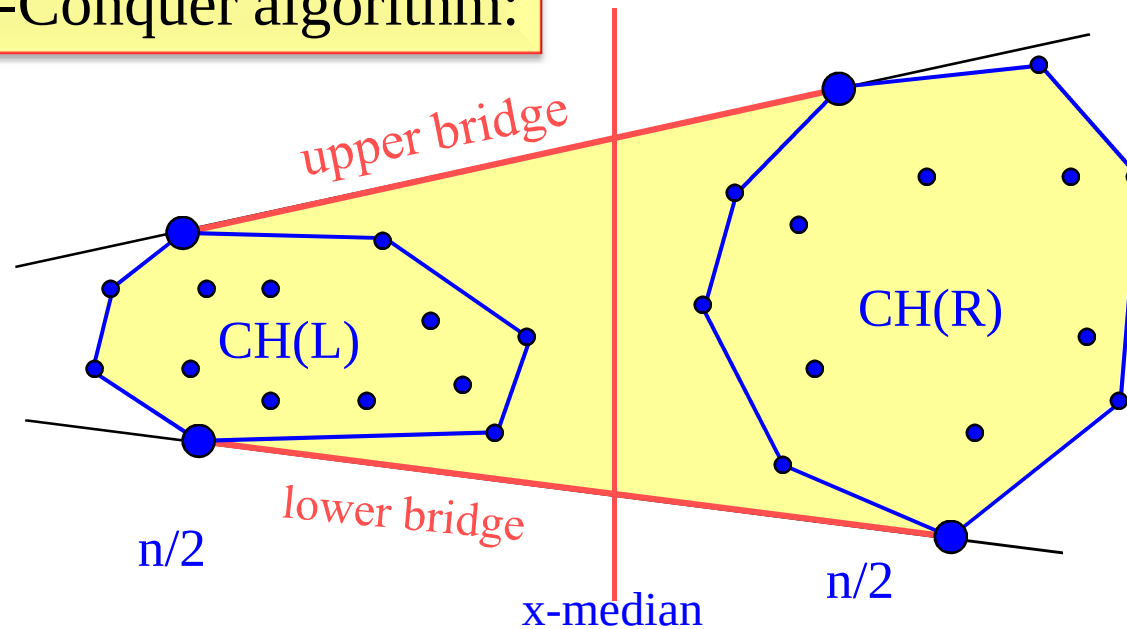
$2T(n/2)$

+

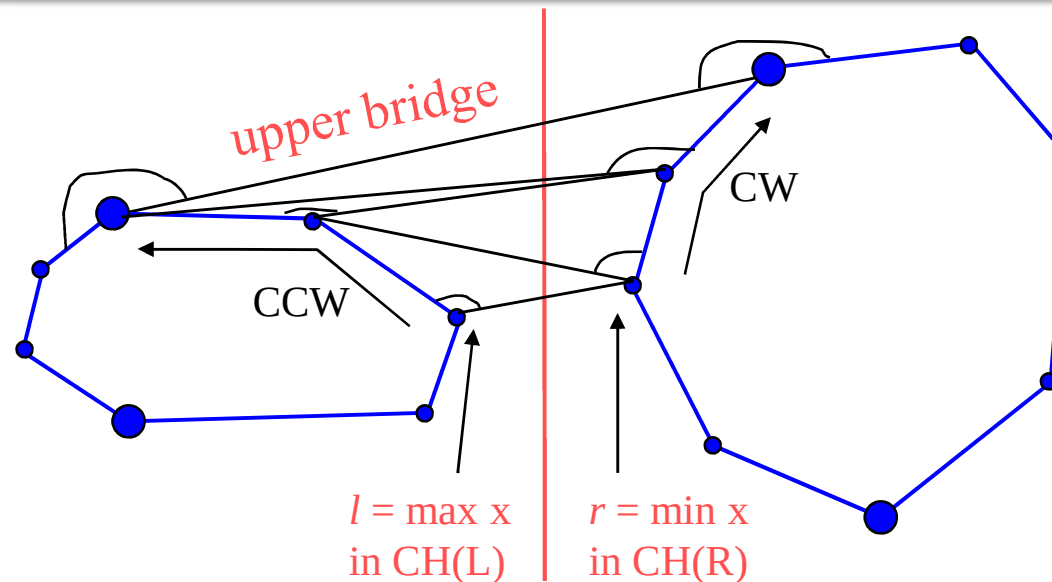
$O(n)$

$$T(n) = 2 T(n/2) + O(n) = O(n \log n).$$

Divide-&-Conquer algorithm:

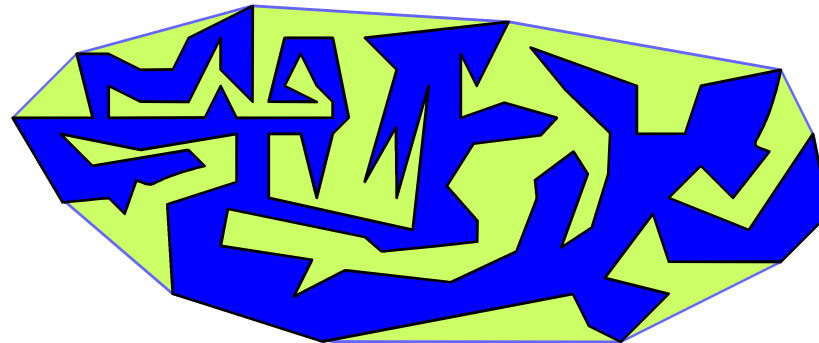


Merge: How to compute the upper-bridge in $O(n)$ time:



Other 2D Convex-Hull Algorithms: (Lecture Notes 7a & 7b)

Melkman [1987]: Convex-Hull of a simple polygon in linear time.



Kirkpatrick-Seidel [1986]:

2D convex-hull of n points in $O(n \log h)$ time.

n = input size

h = output size (i.e., # convex hull vertices)

$3 \leq h \leq n$ (if $n \geq 3$ and not all points are collinear)

[This is a good example of a prune-&-search algorithm.]

Melkman's Linear-Time algorithm for CH of Simple Polygon

Input: Simple polygon $P = (p_1, p_2, \dots, p_n)$

Output: $CH(P)$

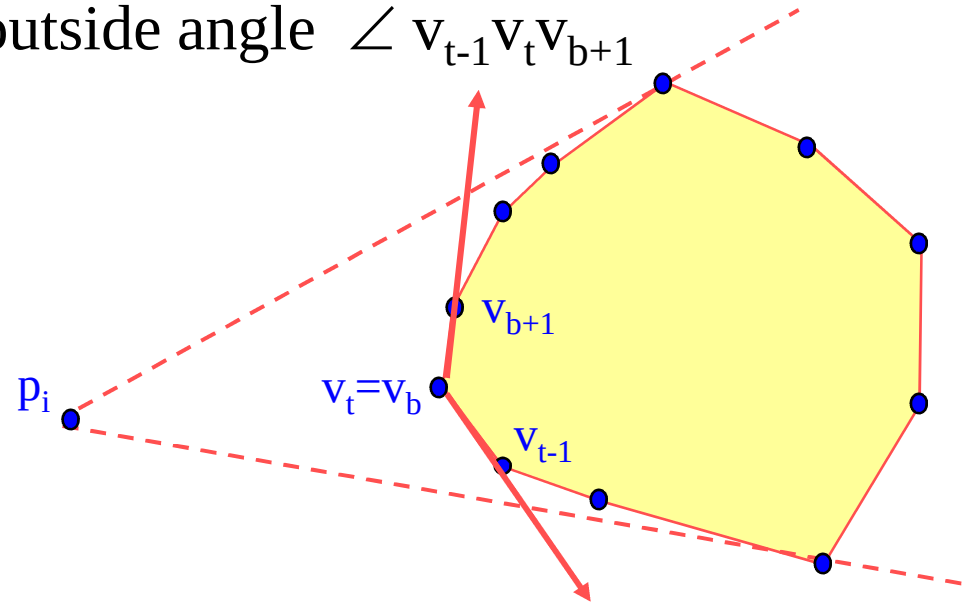
Time: $O(n)$

Method: Incrementally maintain $CH(p_1, p_2, \dots, p_i)$ in a circular deque $D = (v_t, v_{t-1}, \dots, v_{b+1}, v_b)$ (top $v_t = v_b$ bottom)

Algorithm Convex-Hull (P)

```
D ← (p2, p1, p2)      (* CH(p1, p2) *)
for i ← 3 .. n do
    if pi is outside angle  $\angle v_{t-1} v_t v_{b+1}$  then do
        while pi is left of  $\overrightarrow{v_b v_{b+1}}$  do PopBottom(D)
        while pi is right of  $\overrightarrow{v_t v_{t-1}}$  do PopTop(D)
        PushBottom(pi, D)
        PushTop(pi, D)
    end-if
end-for
return D
```

Case 1: p_i is outside angle $\angle v_{t-1}v_tv_{b+1}$

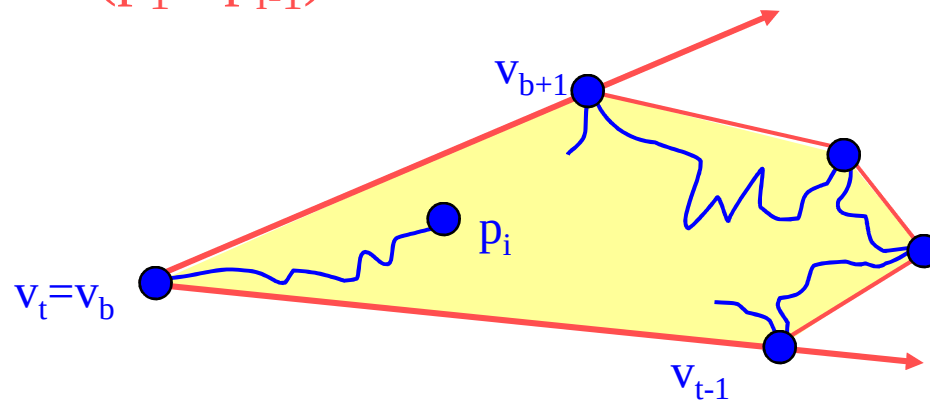


Case 2: p_i is inside angle $\angle v_{t-1}v_tv_{b+1} \Rightarrow p_i$ is not an extreme point

P is simple

$\Rightarrow \exists$ non-crossing chains between v_{t-1} & v_{b+1} and between v_t & p_i

$\Rightarrow p_i \in CH(p_1 \dots p_{i-1})$.

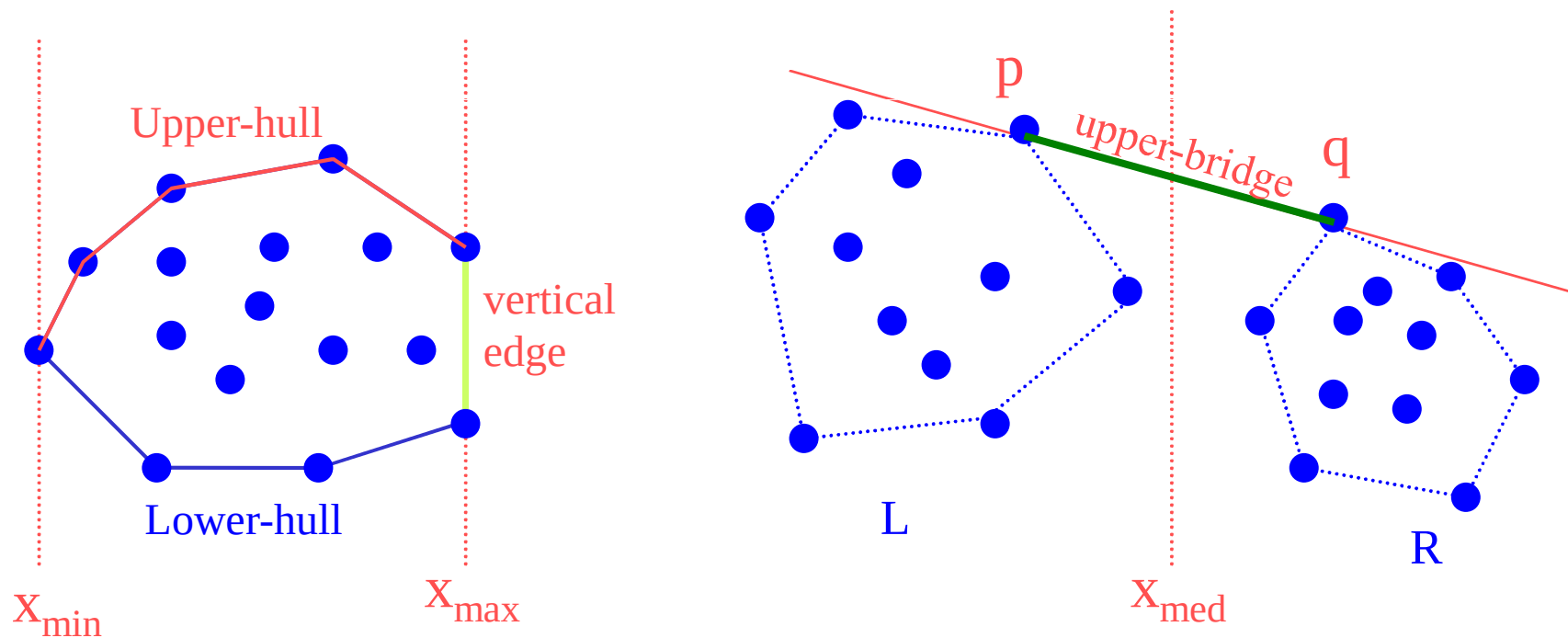


Kirkpatrick-Seidel's $O(n \log h)$ time 2D CH algorithm

IDEA: turn divide-&-conquer into conquer-then-divide!

It finds the upper/lower bridge between the halves L and R in $O(n)$ time, without yet knowing $CH(L)$ and $CH(R)$.

(Must also avoid $O(n \log n)$ time pre-sorting on x-axis.)



How to find Upper-Hull of P:

$P = L \cup R$, $|P| = n$, $|L| \cong |R| \cong n/2$, upper-bridge $\overline{pq}$

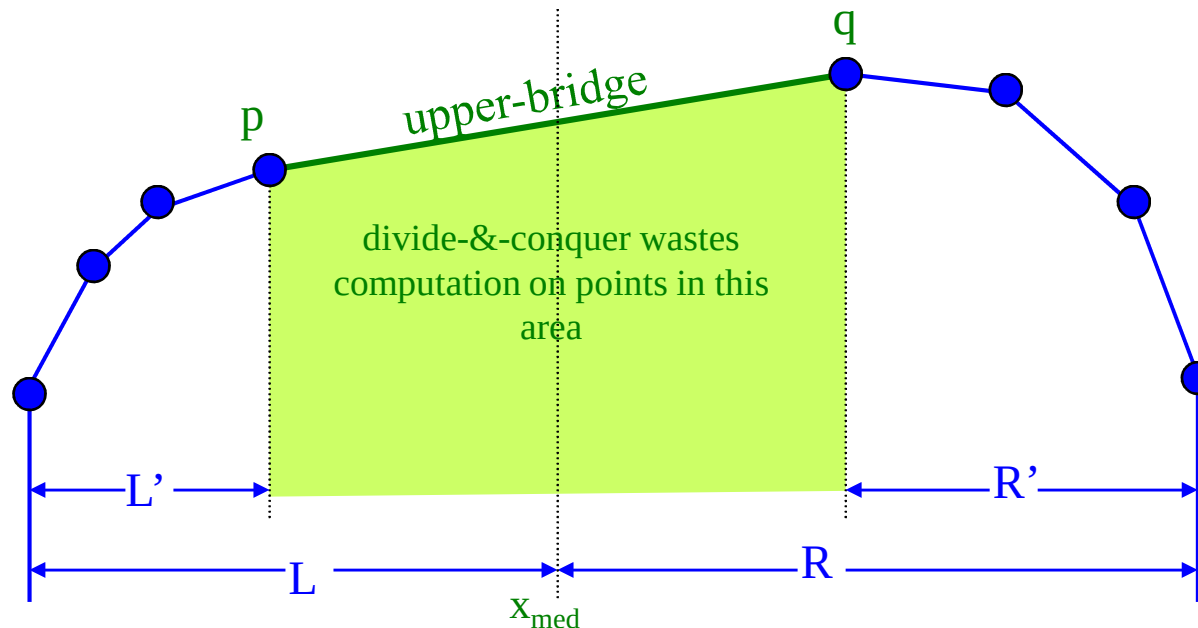
$L' \subseteq L$, $R' \subseteq R$ (points “under” the bridge $\overline{pq}$ ignored)

h = # upper-hull vertices of P

h_1 = # upper-hull vertices of L'

h_2 = # upper-hull vertices of R'

FACT: $h = h_1 + h_2$



$T(n,h)$ **Algorithm** Upper-Hull (P)

$O(1)$ • **if** $|P| \leq 2$ **then return** the obvious answer.

$O(n)$ • $x_{\text{med}} \leftarrow$ median x-coordinates of points in P.
• Partition P into L & R of size $n/2$ each around x_{med} .

$O(?)$ • Find upper-bridge $\overline{pq}$ of L & R, $p \in L, q \in R$.

$O(n)$ • $L' \leftarrow \{ l \in L \mid x_l \leq x_p \}$
• $R' \leftarrow \{ r \in R \mid x_r \geq x_q \}$

$T(n/2, h_1)$ • $\text{LUH} \leftarrow \text{Upper-Hull}(L')$

$T(n/2, h_2)$ • $\text{RUH} \leftarrow \text{Upper-Hull}(R')$

Recursive calls

$O(n)$ • **return** ($\text{LUH}, \overline{pq}, \text{RUH}$) (* upper-hull edge sequence *)

Key idea: compute upper-bridge of L & R in $O(n)$ time (next slides).

Define: $T(n,h)$ = time complexity of Upper-Hull(P)
where $n = |P|$ and $h = \#$ upper-hull vertices.

THEOREM: $T(n,h) = O(n \log h)$.

Proof: Algorithm gives the recurrence:

$$T(n,h) \leq cn + \max \{ T(n/2, h_1) + T(n/2, h_2) \mid h_1 + h_2 = h \} \quad \text{if } h > 2$$

$$T(n,h) \leq cn \quad \text{if } h \leq 2.$$

Induction hypothesis on h : $T(n,h) \leq c n \log 2h, \forall h \geq 1$.

Basis ($h=1,2$): $T(n,h) \leq cn \leq cn \log 2h$.

Induction Step ($h>2$):

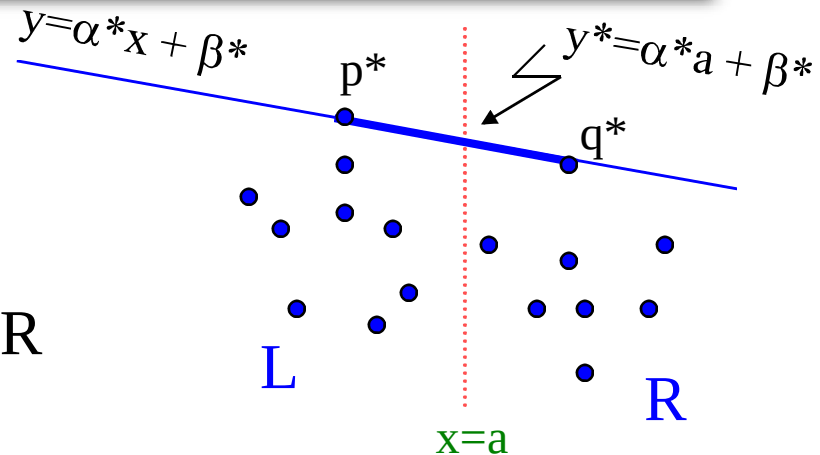
$$\begin{aligned} T(n,h) &\leq cn + \max \{ c(n/2) \log (2h_1) + c(n/2) \log (2h_2) \mid h_1 + h_2 = h \} \\ &= cn + \max \{ c(n/2) \log (2h_1 * 2h_2) \mid 2h_1 + 2h_2 = 2h \} \\ &= cn + c(n/2) \log (h * h) \\ &= cn \log 2h \end{aligned}$$

QED

How to find the **upper-bridge** of L & R in $O(n)$ time?

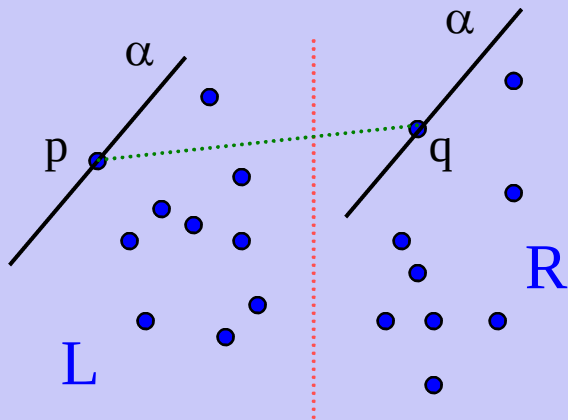
Find $\alpha^*, \beta^* \in \mathbb{R}$ to
minimize $y^* = \alpha^* a + \beta^*$
subject to:

$$x_i \alpha^* + \beta^* \geq y_i \quad \forall p_i \in P = L \cup R$$



This is a 2-variable Linear Program (α^*, β^*).

α^* = slope and β^* = y-intercept of the upper-bridge.



Choose any slope α :

Case 1: $\text{slope}(pq) < \alpha \Rightarrow \alpha^* < \alpha$

Case 2: $\text{slope}(pq) > \alpha \Rightarrow \alpha^* > \alpha$

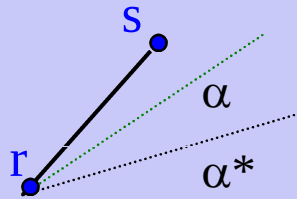
Case 3: $\text{slope}(pq) = \alpha \Rightarrow \alpha^* = \alpha$

p & q = tangency points of α -slope upper-tangents to L & R, respectively

Prune-&-Search on α :

- Partition $L \cup R$ into $n/2$ arbitrary pairs (r,s) , $x_r \leq x_s$
- $\alpha \leftarrow$ median slope of these $n/2$ pair slopes(rs)
- In cases 1 & 2 we can PRUNE $\geq n/4$ points:

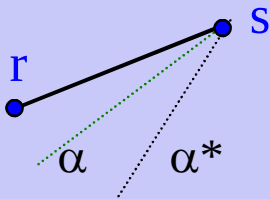
- Case 1: $\text{slope}(rs) \geq \alpha$ (holds for $\geq n/4$ pairs (r,s))



$\Rightarrow \text{slope}(rs) > \alpha^*$

$\Rightarrow$ upper-bridge cannot pass through r. Prune r.

- Case 2: $\text{slope}(rs) \leq \alpha$ (holds for $\geq n/4$ pairs (r,s))



$\Rightarrow \text{slope}(rs) < \alpha^*$

$\Rightarrow$ upper-bridge cannot pass through s. Prune s.

- Case 3: $\text{slope}(pq) = \alpha^*$. We have found the upper-bridge.

Prune-&-Search Bridge Finding:

- Each prune-&-search iteration takes linear time on the points not yet pruned.
- It eliminates at least a quarter of the points (or finds the optimum slope).
- If we start with n points, first iteration eliminates at least $n/4$ points. So, at most $3n/4$ points remain. Then we iterate on the prune-&-search.
- Total bridge-finding time
$$= O(n + (\frac{3}{4}) n + (\frac{3}{4})^2 n + (\frac{3}{4})^3 n + \dots) = O(n).$$

CONCLUSION:

Upper-Bridge (and Lower-Bridge) finding takes $O(n)$ time.

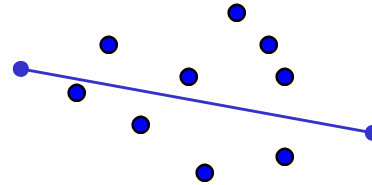
Kirkpatrick-Seidel's CH algorithm takes $O(n \log h)$ time.

Higher Dimensional Convex Hull

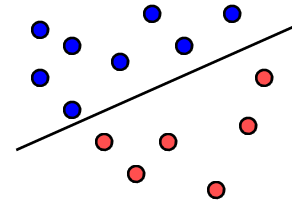
- ❑ Convex hull of n points in 3D can also be computed in $O(n \log n)$ time, e.g., by the divide-&-conquer method.
- ❑ In general, the convex hull of n points in $d > 1$ dimensions can be computed in $O(n \log n + n^{\lfloor d/2 \rfloor})$ time.

Some Applications of Convex Hull

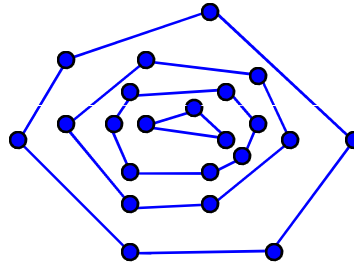
- Diameter $\equiv$ farthest pair



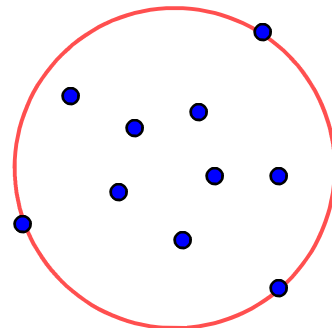
- Linear separability



- Convex Layers:



- Other related problems: **Smallest enclosing circle**
 $O(n)$ time by Megiddo-Dyer's prune-&-search technique



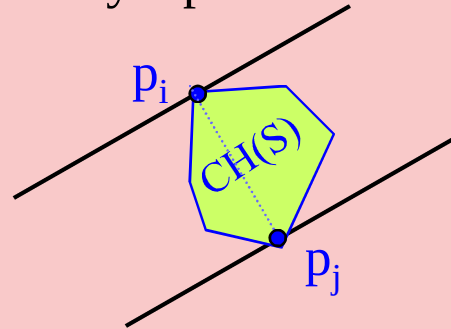
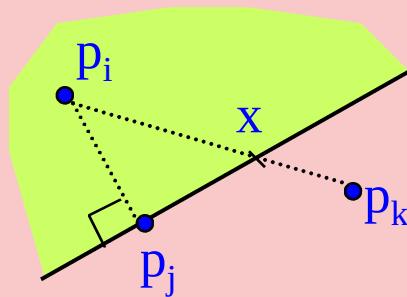
The Farthest Pair among n points

Given a set $S = \{ p_1, p_2, \dots, p_n \}$ on the plane,
Find the farthest pair, i.e., a pair (p_i, p_j) in S such that
$$d(p_i, p_j) \geq d(p_t, p_k) \quad \forall p_t, p_k \in S.$$

Euclidean distance: $d(p_i, p_j) = ((x_i - x_j)^2 + (y_i - y_j)^2)^{1/2}$

FACT: Farthest pair of S is realized by a pair of vertices of $CH(S)$.

Proof:



$d(p_i, p_j) \leq d(p_i, x) < d(p_i, p_k)$, a contradiction.

Algorithm DIAMETER ($p_1, p_2, \dots, p_n$)

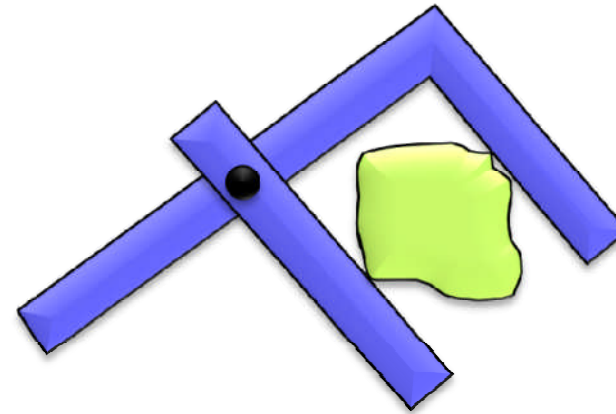
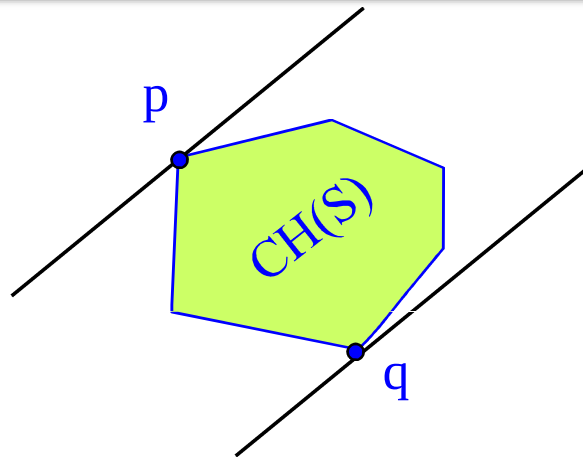
Step 1: $P \leftarrow \text{CH}(p_1, p_2, \dots, p_n)$

$O(n \log n)$

Step 2: Compute diameter of P by the rotating calipers method.

$O(n)$

The rotating calipers method:



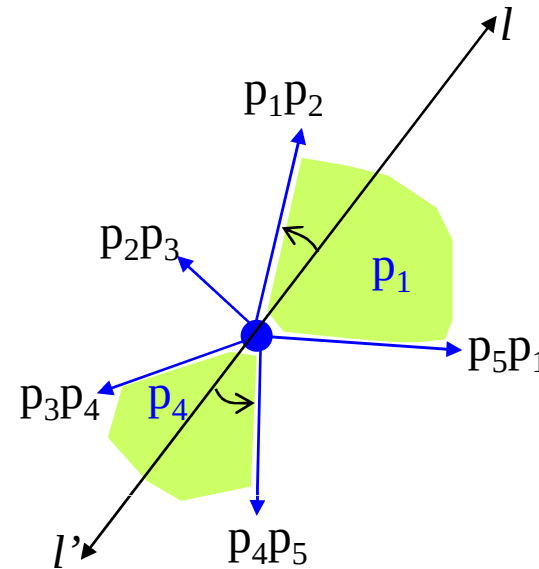
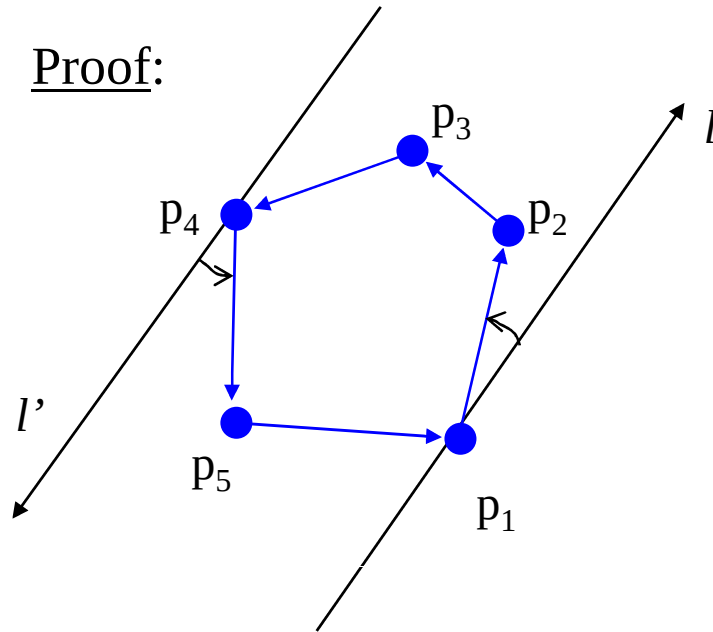
Definition: Antipodal Pair

(p, q) is an antipodal pair if there are a pair of parallel supporting lines of $\text{CH}(S)$ Through p and q that sandwich $\text{CH}(S)$ in between.

FACT: The farthest pair is one of the antipodal pairs.

CLAIM: There are $\leq 2n$ antipodal pairs & they can all be found in $O(n)$ time.

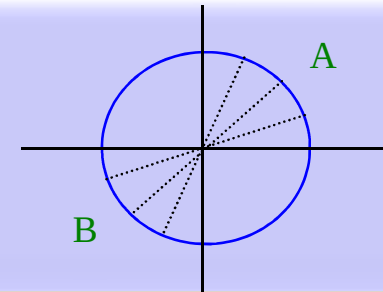
Proof:



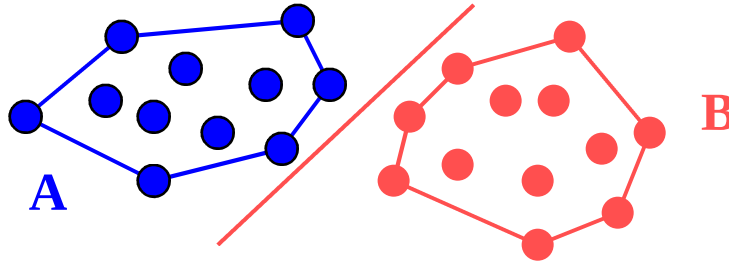
THEOREM:

Farthest pair among n points in the plane can be found in $O(n \log n)$ time.

This is worst-case optimal: $\exists$ reduction from set disjointness



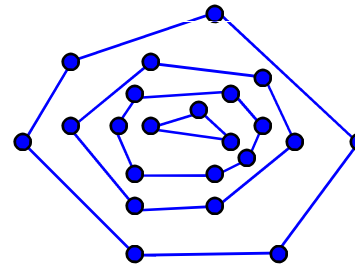
Linear Separability



A & B are linearly separable $\Leftrightarrow$ CH(A) & CH(B) have disjoint interiors
 $O(n \log n)$ time.

Fastest result: $O(n)$ time by prune-&-search [Exercise]

Convex Layers



$O(n^2)$ time: by repeated application of Jarvis' March.

Fastest result: $O(n \log n)$ time

Chazelle[1985], "On the convex layers of a planar set",

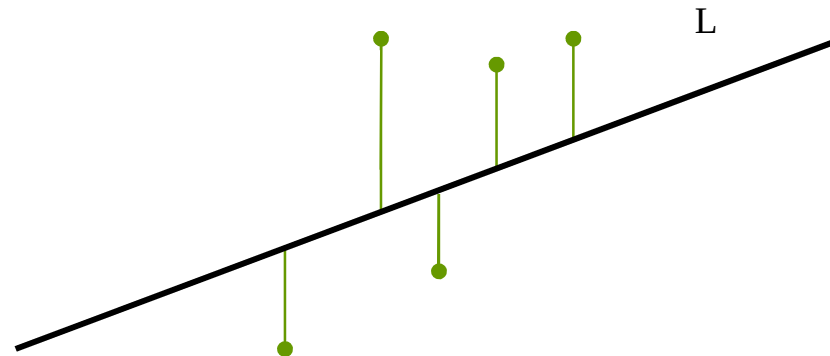
IEEE Transactions on Information Theory, vol IT-31, No. 4, pp: 509-517.

Exercises

1. **[CLRS, Exercise 33.1-5, pages 1020-1021]** Prove or disprove: a sequence $P = \langle p_0, p_1, \dots, p_{n-1} \rangle$ of n points in the plane forms the boundary of a convex polygon if and only if, for $i = 0..n-1$ (mod n index arithmetic), $\Delta(p_{i-1}, p_i, p_{i+1})$ are all positive or all negative, i.e., either all are left turns or all are right turns.
2. **Convex hull of sorted points:** Show that the convex hull of a set P of n points in the plane can be computed in $O(n)$ time if the points in P are already sorted on their x -coordinates. Prove the correctness and time complexity of your algorithm.
3. **[CLRS, Exercises 33.3-5 & 33.3-6, page 1039] On-line convex hull:** We are given a set P of n points in the plane, one point at a time on-line. Update $CH(P)$ after receiving each point.
 - (a) Design and analyze a simple $O(n^2)$ time algorithm for this problem.
 - (b) Design and analyze an $O(n \log n)$ time algorithm for this problem.
4. **Convex hull in integer grid:** Given a planar set P of n points with coordinates as positive integers at most n^d , where d is some constant, show $CH(P)$ can be constructed in $O(n)$ time.
5. **Convex hull of unit circles:** Let S be a set of n given, possibly intersecting, unit circles (i.e., all with radius 1) on the plane. The boundary of $CH(S)$ consists of line-segments and some circular arcs that are pieces of circles in S .
 - (a) Show that each circle can contribute at most one circular arc to the boundary of $CH(S)$.
 - (b) Let C be the set of n centers of the circles in S . Show that circle $A \in S$ contributes an arc to the boundary of $CH(S)$ if and only if the center of A is an extreme point of C .
 - (c) Design and analyze an efficient algorithm that outputs the boundary of $CH(S)$. [The output should be in the form of a finite closed chain of line segments and circular arcs.]

6. **[CLRS, Exercise 3.1-4, page 1020]** Show how to determine in $O(n^2 \log n)$ time whether any 3 points in a set of n points in the plane are collinear.
7. Given a set P of n points in the plane, construct a simple polygon with P as its vertex set.
 - (a) Show the $\Omega(n \log n)$ lower bound on the worst-case time complexity of this problem.
 - (b) Design an $O(n \log n)$ time algorithm for this problem.
8. Show that any set S of at least $d+2$ points in $\mathbb{R}^d$ can be partitioned into two nonempty subsets A and B such that $\text{CH}(A) \cap \text{CH}(B) \neq \emptyset$. **[Hint: consider linear combinations of points in S .]**
9. Given a set P of n points and another point q , all in the plane, design an $O(n)$ time algorithm to decide whether q is in $\text{CH}(P)$. **[Note: $\text{CH}(P)$ is not part of the input.]**
10. We are given a set P of n points, and a triangular area T specified by its three vertices $T = (t_1, t_2, t_3)$, all in the plane.
 - (a) Design and analyze an $O(n)$ -time algorithm to decide whether $\text{CH}(P)$ (interior as well as boundary of convex hull of P) intersects with triangle area T .
 - (b) Consider the modified version of the problem in part (a), where we want to decide whether the boundary of $\text{CH}(P)$ intersects area T . Design and analyze an efficient algorithm for this problem.

11. **Polygon distance:** Given two disjoint convex polygons P and Q in the plane, design an efficient algorithm to determine the closest pair of points between P and Q . (Note these points may not necessarily be vertices.)
12. **Linear & Circular Separability:** Use prune-&-search to show the **linear separability** problem can be solved in $O(n)$ time. How about **circular separability** ($\exists$ a circle that contains one set inside, the other outside)?
13. **Linear Regression:** Given a set P of n points in the plane, determine a line L such that the largest vertical distance between any point of P to L is minimized. Show an $O(n)$ time solution to this problem using prune-&-search.



END